Geodesic Distances: the Mathematics of `geodesic.py`

November 2026

This note explains how `geodesic.py` (in `Dubins_TSP_New_Implementation_claude/`) computes the obstacle-aware distances of the paper: r_i of eq. (9), the matrix D_{geo} of Step 1 of Section 0.9, and the obstacle-free paths that Step 2 of Section 0.11 builds on. It states every formula the code uses, in the notation of `Dubins_TSP.pdf`. Section 1 gives the idea in a few lines. Sections 3–6 give the formulas step by step, Section 7 says what the result is and is not, and Section 8 says how each statement was checked. Table 3, in the appendix, lists every symbol with its name in the code.

1 The Idea in Brief

A sink cannot drive through an obstacle, so when an obstacle is in the way the straight-line distance between two points is too short. The *geodesic distance* is the length of the shortest path that goes round the obstacles. `geodesic.py` finds it in four steps (Figure 1):

1. **Grow the obstacles** (`inflate`). Each obstacle is enlarged by a safety margin, so a path that stays outside the grown obstacle keeps about that distance from the real one.
2. **Decide which straight lines are allowed** (`_inside`). A straight line is allowed if it does not cut into a grown obstacle. Touching it, or running along its edge, is allowed.
3. **Build a graph** (`build`). Its *nodes* are the base, the sensors and the corners of the grown obstacles. Two nodes are joined by an *edge* if the straight line between them is allowed, and the edge's length is its *weight*.
4. **Find shortest paths** (`build`, `path`). Dijkstra's algorithm [2] finds the shortest chain of edges from each terminal (the base or a sensor) to every other terminal. Its length is the geodesic distance, and the chain itself is the obstacle-free path.

Why corners are enough: a shortest path round polygons is like a string pulled tight. It consists of straight pieces that bend only at corners of the obstacles [1, 3]. So the graph contains the shortest path, and no other bending points are needed.

One detail differs from the textbook method: the code uses two margins. A line between two corners must stay outside the obstacles grown by 7 m, but a line that touches the base or a sensor only outside those grown by 2.5 m (Section 2).

2 Obstacles, Sensors and Margins

The field contains m polygonal obstacles $O_1, \dots, O_m$, the base station $S_0 = (x_0, y_0)$ and the sensors $S_i = (x_i, y_i)$, $i = 1, \dots, n$. The base and the sensors together are called the $n_t = n + 1$ *terminals*, numbered $0, 1, \dots, n$ as in the paper. In the code, terminal 0 is the base and terminal i is `scen["xy"][i - 1]`, the sensor with the 0-based id $i - 1$.

The paper defines the free space by growing the obstacles by a safety clearance, eqs. (4) and (5):

$$F = \mathbb{R}^2 \setminus \bigcup_{j=1}^{m} \operatorname{int}\big(O_j^{\text{inflated}}\big), \qquad r_{\text{safe}} = \frac{w}{2} + \delta_{\text{margin}}. \tag{1}$$

`geodesic.py` uses two larger margins built on r_{safe}:

$$r_{\text{safe}} = 2\,\text{m}, \qquad r_{\text{term}} = r_{\text{safe}} + 0.5\,\text{m} = 2.5\,\text{m}, \qquad r_{\text{plan}} = r_{\text{safe}} + R = 7\,\text{m}, \tag{2}$$

with the turning radius $R = L/\tan\delta_{\max} = 5\,\text{m}$ of eq. (7) and the defaults of `config.py`.

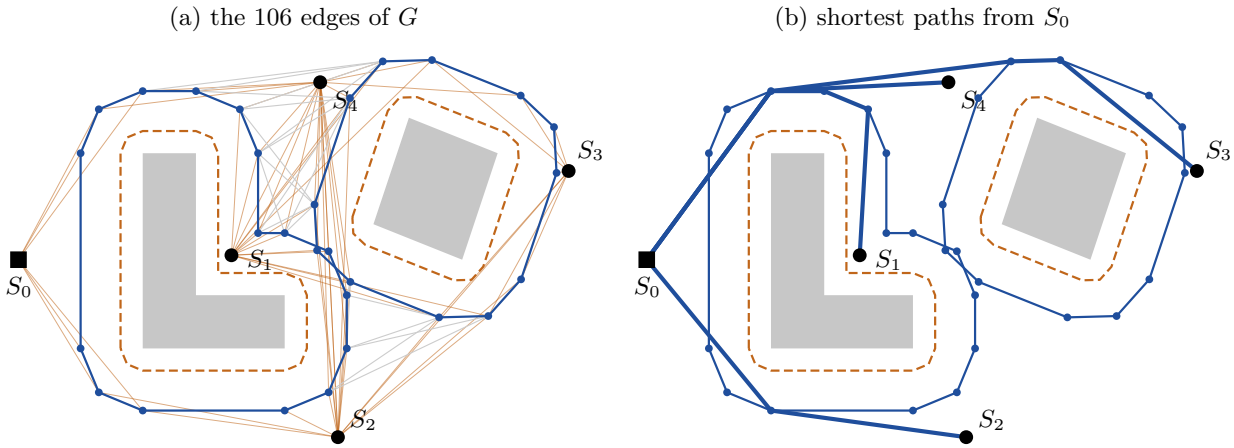


Figure 1: A small example, computed by `geodesic.build`: two obstacles (grey), the base S_0 (square) and four sensors. The blue polygons are the obstacles grown by $r_{\text{plan}} = 7\,\text{m}$; their 28 corners (blue dots) are graph nodes. The dashed orange polygons are the obstacles grown by $r_{\text{term}} = 2.5\,\text{m}$. (a) Of the 528 pairs of nodes, 106 are joined: 61 lines that touch the base or a sensor, allowed because they stay out of the orange polygons (orange), and 45 lines between two corners, allowed because they stay out of the blue polygons (grey). (b) The shortest path from S_0 to each sensor. S_1 sits in the inside corner of the L, within the blue polygon, so only a line tested against the orange polygon can reach it.

- r_{plan} applies to lines between two obstacle corners. The extra R leaves the sink room to turn where the path bends at a corner.
- r_{term} applies to lines that start or end at a terminal. Sensors may lie closer to an obstacle than r_{plan} (on seeds 1–10, 577 of the 1 015 sensors do), and they could never be reached if every line to them needed 7 m. The code has no name for r_{term}; it writes `p["r_safe"] + 0.5`.
- r_{safe} itself is not used in `geodesic.py`. The clearance of the final trajectory is checked against it later (Section 7).

Table 1 lists every constant of the file.

Table 1: Constants in `geodesic.py`, with the defaults of `config.py`.

Constant	Value	Role
$r_{\text{plan}} = r_{\text{safe}} + R$	7 m	growth for the corner nodes and for lines between two corners
$r_{\text{term}} = r_{\text{safe}} + 0.5$	2.5 m	growth for lines that start or end at a terminal
q (`quad_segs`)	2	sets the chord size at the corners of a grown obstacle, eq. (4)
ε	10^{-6} m	how far a line may enter a grown obstacle, eq. (8)
`require_all`	`True`	stop if a terminal cannot be reached (`scenario.py` passes `False`)

3 Step 1: Growing the Obstacles

In words: `inflate` replaces each obstacle by a polygon that is larger by about r all round.

The exact grown obstacle is the set of points within r of the obstacle:

$$G_r(O_k) = \{x \in \mathbb{R}^2 : \text{dist}(x, O_k) \le r\}. \tag{3}$$

Its corners are round. The code uses a polygon $P_r(O_k)$ in its place, made by shapely's `buffer` with `quad_segs` $= q = 2$. Its boundary follows four rules (Figure 2).

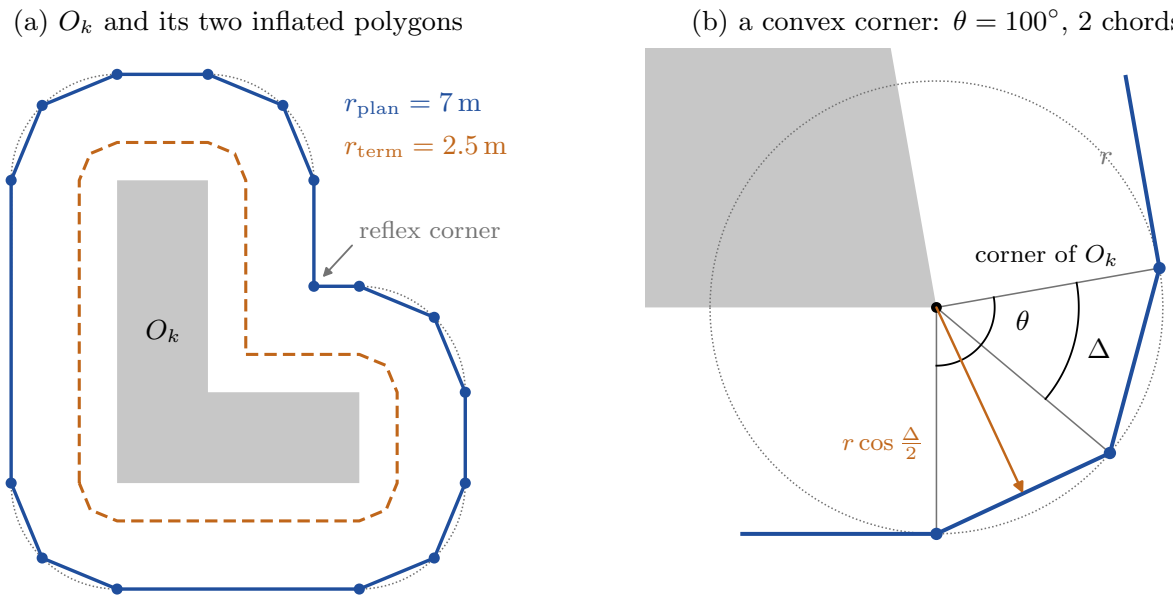


Figure 2: (a) An L-shaped obstacle (16 m by 20 m, arms 6 m wide) grown by 7 m (blue; its 12 corners become graph nodes) and by 2.5 m (dashed orange), as `geodesic.inflate` makes them. The dotted line is the exact grown obstacle of eq. (3), with round corners. At the inside corner of the L the two moved edges simply meet. (b) One outer corner, where the boundary turns through $\theta = 100°$. Eq. (4) gives 2 chords of $\Delta = 50°$. Their ends lie on the circle of radius r, and their midpoints are $r\cos 25° = 0.906\,r$ from the corner.

1. Edges. Each edge of O_k is moved outward, parallel to itself, by r.

2. Outer (convex) corners. Here the round arc of G_r is replaced by straight chords. Let θ, between 0 and π, be the angle through which the boundary turns at the corner c: the angle between the outward normals of the edge arriving at c and the edge leaving it. The arc is cut into

$$n_\theta = \max\Big(1,\ \Big\lfloor \frac{4\theta}{\pi} + \frac{1}{2} \Big\rfloor\Big) \quad \text{equal chords, each turning through} \quad \Delta = \frac{\theta}{n_\theta}. \tag{4}$$

In words: the arc is cut into pieces of about 45° (that is, $\pi/(2q)$ with $q = 2$), with the number rounded to the nearest whole number but at least one. A right-angled corner gets two chords of 45°. The chord ends lie on the circle of radius r about the corner:

$$c + r\,\big(\cos(\varphi_0 + j\Delta),\ \sin(\varphi_0 + j\Delta)\big), \qquad j = 0, 1, \ldots, n_\theta, \tag{5}$$

where φ_0 is the direction of the outward normal of the arriving edge, and the angle grows in the direction in which the boundary turns.

3. Inner (reflex) corners. At a corner such as the inside corner of an L, the two moved edges cross, and the crossing point is the corner of the polygon.

4. Overlaps. Where the moved boundary overlaps itself, as in the pocket of a C or the notches of a cross, the polygon is the union of the pieces, so the covered parts of the boundary disappear.

How close the polygon comes to the obstacle. A chord lies inside the circle, so it comes closer to the corner than r: its midpoint is $r\cos(\Delta/2)$ away. The worst case is the largest Δ. By eq. (4), a corner gets a single chord only if $\theta < 67.5°$, and $n \geq 2$ chords only if $\theta < (n + \frac{1}{2}) \cdot 45°$, so

$$\Delta < 67.5° = \frac{3\pi}{8}, \qquad r\cos\frac{\Delta}{2} > r\cos\frac{3\pi}{16} = 0.8315\,r. \tag{6}$$

No vertex is farther than r from the obstacle. Together with eq. (6), every point x on the boundary of the polygon satisfies

$$\boxed{0.8315\,r \;\le\; \operatorname{dist}(x, O_k) \;\le\; r.} \tag{7}$$

Equivalently, $P_r(O_k)$ contains $G_{0.8315\,r}(O_k)$ and lies inside $G_r(O_k)$. So the polygon grown by $r_{\text{term}} = 2.5\,\text{m}$ keeps at least 2.08 m from the obstacle, more than $r_{\text{safe}} = 2\,\text{m}$; the polygon grown by $r_{\text{plan}} = 7\,\text{m}$ keeps at least 5.82 m; and since $5.82 > 2.5$, the 2.5 m polygon lies inside the 7 m polygon.

4 Step 2: Which Straight Lines Are Allowed

In words: a line is allowed if it does not cut into a grown obstacle; a tiny tolerance lets it touch.

The function `_inside` takes the inside of the grown obstacles, shrunk by $\varepsilon = 10^{-6}$ m:

$$I_r = \big\{x : \text{for some } k,\ x \text{ and every point within } \varepsilon \text{ of it lie in } P_r(O_k)\big\}. \tag{8}$$

A straight line from p to q is *allowed at margin* r if it has no point in I_r:

$$[p, q] \cap I_r = \emptyset. \tag{9}$$

Why shrink: shapely counts touching as meeting. Without the shrinking, a line running along the edge of a grown obstacle would be refused, and so would every line from a corner node, because the corner lies on the edge. With it, a line may touch the polygon, run along its edge, or enter it by at most ε. By eq. (7), an allowed line keeps at least $0.8315\,r - \varepsilon$ from every obstacle.

5 Step 3: The Graph

In words: the nodes are the terminals and the corners of the 7 m polygons; two nodes are joined if the line between them is allowed at the margin that applies to it.

Nodes (`build`). The terminals come first, then the corners:

$$\begin{aligned} &v_a = S_a \quad (a = 0, \dots, n), \\ &v_{n_t}, \dots, v_{N-1} = \text{the vertices of the outer boundaries of } P_{r_{\text{plan}}}(O_1), \dots, P_{r_{\text{plan}}}(O_m). \end{aligned} \tag{10}$$

If a polygon had a hole, the corners of the hole would not be nodes.

Edges. Every pair of nodes $a < b$ is tested once. Because the terminals come first, the line touches a terminal exactly when $a < n_t$:

$$\boxed{a \text{ and } b \text{ are joined} \iff \begin{cases} [v_a, v_b] \cap I_{r_{\text{term}}} = \emptyset & \text{if } a < n_t \text{ (the line starts or ends at a terminal),} \\ [v_a, v_b] \cap I_{r_{\text{plan}}} = \emptyset & \text{if } a \ge n_t \text{ (the line joins two corners).} \end{cases}} \tag{11}$$

If there are no obstacles, every pair is joined. The weight of an edge is its length,

$$w_{ab} = \|v_a - v_b\| = \sqrt{(x_a - x_b)^2 + (y_a - y_b)^2}, \tag{12}$$

and the graph is undirected: the edge can be used both ways. With N nodes, $N(N-1)/2$ pairs are tested. On seed 1 there are 99 terminals and 148 corner nodes: 30 381 pairs, of which 7 898 are joined (6 800 touching a terminal and 1 098 between two corners).

6 Step 4: Shortest Distances and Paths

In words: the geodesic distance is the length of the shortest chain of edges, and Dijkstra's algorithm finds it.

Geodesic distance. For terminals a and b,

$$\boxed{D_{ab} = \min\Big\{\sum_{l=1}^{L} w_{u_{l-1}u_l} \;:\; u_0 = a,\; u_L = b,\; \text{each } u_{l-1}, u_l \text{ joined}\Big\},} \tag{13}$$

the smallest total length of a chain of edges $a = u_0, u_1, \ldots, u_L = b$. If no chain exists, $D_{ab} = \infty$.

Dijkstra's algorithm (`dijkstra`), run once from each terminal $a = 0, \ldots, n$. It spreads out from a in order of distance, like a wave. It starts with $d_a(a) = 0$ and $d_a(v) = \infty$ for every other node. Then, again and again, it fixes the nearest node u not yet fixed, and checks whether going through u shortens the path to each neighbour v:

$$\text{if } d_a(u) + w_{uv} < d_a(v): \qquad d_a(v) \leftarrow d_a(u) + w_{uv}, \qquad \text{pred}_a(v) \leftarrow u. \tag{14}$$

$\text{pred}_a(v)$ remembers the node before v on the best path found so far. When the algorithm stops, each reachable node's distance is the best "distance of a neighbour plus the last edge":

$$d_a(v) = \min_{u \text{ joined to } v} \big(d_a(u) + w_{uv}\big) \quad (v \neq a), \tag{15}$$

and $D_{ab} = d_a(b)$. The code keeps $D = (D_{ab})$, an $n_t \times n_t$ matrix over the terminals.

The paper's distances. Eq. (9) and Step 1 of Section 0.9 are entries of D:

$$r_i = d_{\text{geo}}(S_0, S_i) = D_{0i}, \qquad D_{\text{geo}} = \big[d_{\text{geo}}(i,j)\big] = \big[D_{ij}\big]_{i,j=1,\ldots,n}. \tag{16}$$

In the code these are `geo["D"][0, 1:]` and `geo["D"][1:, 1:]`, with 0-based sensor ids. In Figure 1, S_1 is 24.0 m from the base in a straight line, but the L is in the way, and the path over the top of the L gives $D_{01} = 51.4$ m. For the other sensors D_{0i} is 44.2, 76.2 and 43.6 m, against straight-line distances of 41.2, 62.8 and 39.5 m.

The path (`path`). Follow the predecessors back from b until a is reached, then read the nodes forward:

$$u_L = b, \qquad u_{l-1} = \text{pred}_a(u_l) \;\text{ until } u_0 = a, \qquad \Gamma_{ab} = \big(v_{u_0}, v_{u_1}, \ldots, v_{u_L}\big). \tag{17}$$

Γ_{ab} is a chain of straight lines from S_a to S_b of length D_{ab}, defined only when $D_{ab} < \infty$. `trajectory.py` uses its nodes as the waypoints of a leg.

Unreachable terminals. If some $D_{ab} = \infty$, `build` stops with the error "some sensors cannot be reached from the base station". `scenario.py` calls it with `require_all=False` instead, to drop candidate sensors that cannot be reached.

7 What D Is, and What It Is Not

It behaves like a distance. Because D is a shortest-path length in an undirected graph with lengths as weights,

$$D_{aa} = 0, \qquad D_{ab} = D_{ba}, \qquad D_{ac} \le D_{ab} + D_{bc}, \qquad D_{ab} \ge \|S_a - S_b\|. \tag{18}$$

That is: zero from a point to itself, the same both ways, never longer than going via a third terminal, and never shorter than the straight line.

It lies between the true distances at the two margins. If every line were tested at one margin, D would be the exact length of the shortest path round the obstacles grown by that margin, because such a

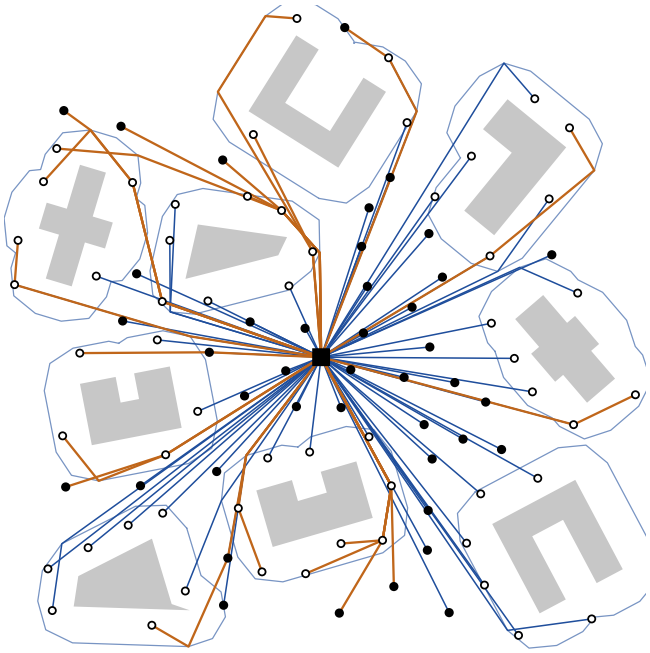


Figure 3: Seed 1 with the default settings: 9 obstacles (grey), the obstacles grown by 7 m (thin blue outline), the base (square) and 98 sensors. The 57 hollow sensors lie inside the 7 m outline. The lines are the shortest paths Γ_{0i} from the base. The 23 orange paths pass through another sensor and bend there with only the 2.5 m margin; the blue ones bend only at corners of the 7 m polygons. Some paths go round obstacles that reach beyond the 100 m map.

path bends only at corners [1, 3]. With two margins, D lies between the two exact lengths. Write $d(p, q; X)$ for the length of the shortest path from p to q that has no point in the set X. Then

$$\boxed{d\big(S_a, S_b;\ I_{r_{\text{term}}}\big)\ \le\ D_{ab}\ \le\ d\Big(S_a, S_b;\ \operatorname{int}\textstyle\bigcup_k P_{r_{\text{plan}}}(O_k)\Big),} \tag{19}$$

where the right-hand side needs S_a and S_b to lie outside the 7 m polygons. In words: D is never shorter than the shortest path that stays out of the 2.5 m polygons, and never longer than the shortest path that stays out of the 7 m polygons. The lower bound holds because every edge is allowed at 2.5 m; a line allowed at 7 m is also allowed at 2.5 m, since the 2.5 m polygon lies inside the 7 m one. The upper bound holds because the shortest path that stays out of the 7 m polygons bends only at their corners, which are nodes, and each of its lines passes the test of eq. (11).

Paths can pass through other sensors. The terminals are nodes too, so a chain in eq. (13) may pass through another sensor. Both of its lines at that sensor are tested only at 2.5 m, so a sensor close to an obstacle can act as a tighter corner than the corners of the 7 m polygons (Figure 3). On seeds 1–10, 245 of the 1 015 shortest paths from the base pass through another terminal.

What is not guaranteed here. D and Γ_{ab} describe chains of straight lines, not the Dubins paths the sinks drive. An allowed line keeps $0.8315\, r_{\text{term}} - \varepsilon \ge 2.08$ m from the obstacles if it touches a terminal, and $0.8315\, r_{\text{plan}} - \varepsilon \ge 5.82$ m if it joins two corners. But a Dubins path that follows the waypoints can swing wider or cut inside. So eq. (19) of the paper is checked on the final trajectory Γ_k instead, against the original obstacles, whose union `build` also returns:

$$O = \bigcup_{k=1}^{m} O_k, \qquad \operatorname{dist}\big(\Gamma_k(t), O\big) \ge r_{\text{safe}}. \tag{20}$$

8 How the Statements Were Checked

`check_formulas.py`, next to the source of this note, repeats every check below.

- **Chords at the corners**, eqs. (4)–(7). 3 000 random obstacles made by `scenario.py` (random polygons, L, C and cross shapes) were grown by 2.5 m and by 7 m. All 29 047 outer corners whose arc lies on the boundary have exactly the vertices of eq. (5). At the other 6 119, another arm of the same obstacle covers part of the arc (rule 4). The largest chord angle was 67.44°, the closest approach of a boundary to its obstacle $0.8318\,r$, and the farthest vertex lay at r to within $10^{-11}r$.
- **Holes.** None of the 108 polygons grown by 7 m on seeds 1–10 has a hole, so no corner is left out of eq. (10).
- **Distance properties**, eq. (18). On seeds 1–10, D was symmetric and met the triangle inequality to within 6×10^{-14} m, and never fell below the straight-line distance.
- **Bounds**, eq. (19). On seeds 1–10, both bounds held for every pair of terminals, with the exact distances computed by visibility graphs tested at one margin only.
- **Counts.** On seeds 1–10, 577 of 1 015 sensors lie inside the 7 m polygons, and 245 of the 1 015 paths from the base pass through another terminal.

9 Relation to the Paper

The paper says what d_{geo} is (eq. (9): "an obstacle-aware geodesic walking distance in the free space"; Step 1 of Section 0.9: "the shortest obstacle-respecting distance"; and "visibility-based" geometry in the introduction of Section 0.9), but not how to compute it. The code fills the gaps as in Table 2; the README of the implementation records the same choices in its table "Choices where the paper is silent".

Table 2: The paper and `geodesic.py`.

Topic	Paper	`geodesic.py`
Margin	r_{safe}, eqs. (4)–(5)	r_{plan} and r_{term} in the graph, eq. (11); r_{safe} on the final path, eq. (20)
Shape of a grown obstacle	not given	polygon with chords at the corners, eqs. (4)–(7)
How d_{geo} is computed	"visibility-based"	visibility graph and Dijkstra, eqs. (10)–(13)
Sensors near obstacles	not covered	lines that touch a terminal use r_{term}, eq. (11)
Paths via sensors	not covered	allowed, eq. (13)
Unreachable sensors	not covered	an error, or dropped by `scenario.py`

References

[1] T. Lozano-Pérez and M. A. Wesley, "An algorithm for planning collision-free paths among polyhedral obstacles," Communications of the ACM, vol. 22, no. 10, pp. 560–570, 1979.

[2] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, pp. 269–271, 1959.

[3] M. de Berg, O. Cheong, M. van Kreveld and M. Overmars, Computational Geometry: Algorithms and Applications, 3rd ed., Springer, 2008, Chapter 15 (Visibility Graphs).

A Symbols

Table 3: Symbols and their names in `geodesic.py`.

Symbol	Code	Meaning
O_k, $k = 1, \ldots, m$	`scen["obstacles"][k - 1]`	obstacle polygon (its vertices)
S_0	`scen["base"]`, terminal 0	base station
S_i, $i = 1, \ldots, n$	`scen["xy"][i - 1]`, terminal i	sensor
n_t	`n_terminals`, `n_t`	number of terminals, $n + 1$
$r_{\text{safe}}, R, r_{\text{plan}}$	`p["r_safe"], p["R"], p["r_plan"]`	margins and turning radius, eq. (2)
r_{term}	`p["r_safe"] + 0.5`	margin of lines that touch a terminal
$G_r(O_k)$	—	exact grown obstacle, eq. (3)
$P_r(O_k)$	`inflate(obstacles, r)`	grown obstacle as a polygon (Section 3)
q	`quad_segs=2`	sets the chord size at the corners
ε	`1e-6`	how far a line may enter a grown obstacle
I_r	`_inside(inflate(obstacles, r))`	inside of the grown obstacles, shrunk by ε
$v_0, \ldots, v_{N-1}$	`nodes`	graph nodes: the terminals, then the corners of the 7 m polygons
w_{ab}	`graph`	edge lengths
$d_a(v)$, $\text{pred}_a(v)$	`dist[a, v]`, `pred[a, v]`	shortest distance from terminal a to node v, and the node before v
D	`D`	geodesic distances between terminals, an $n_t \times n_t$ matrix
Γ_{ab}	`path(geo, a, b)`	shortest obstacle-free chain of lines from terminal a to terminal b
O	`obstacles`	union of the original obstacles, for clearance checks